```
    }

    public double getWidth() {
        return this.width;
    }

    public void setWidth(double width) {
        if (width <= 0) {
            throw new IllegalArgumentException();
        }
        this.width = width;
    }

    @Override
    public String toString() {
        return "Rectangle{" + "length=" + length + ", width=" + width +
'}';
    }

    @Override
    public int hashCode() {
        return Double.hashCode(length) + Double.hashCode(width);
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Rectangle other = (Rectangle) obj;
        if (Double.compare(width, other.width) != 0) {
            return false;
        }
        return Double.compare(length, other.length) == 0;
    }

    public boolean aproximateEquals(Rectangle r) {
        if (equals(r)) {
            return true;
        }
        return Math.abs(width - r.width) < 0.001 && Math.abs(length -
r.length) < 0.001;
    }
}
```

Κώδικας 11.8 *Υπόδειγμα ολοκληρωμένης κλάσης*

11.7 Απαριθμήσιμοι τύποι

Μερικές φορές στα προγράμματά μας χρειαζόμαστε ένα ή περισσότερα σύνολα από σταθερές. Για παράδειγμα, έστω ότι θέλουμε να αναπαραστήσουμε με σταθερές κάθε ημέρα της εβδομάδας έτσι ώστε να μην αναφερόμαστε σε κυριολεκτικούς ακέραιους που κάνουν το πρόγραμμά μας δυσανάγνωστο.
Για την καλύτερη ικανοποίηση τέτοιων αναγκών, η Java προσφέρει τον τύπο enum. Στον κώδικα 11.9, παρουσιάζουμε τη δήλωση ενός απαριθμήσιμου τύπου ορισμού των ημερών της εβδομάδας.

```
public enum Day {
    MONDAY,
    THUSDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY;
}
```

Κώδικας 11.9 *Απαριθμήσιμος τύπος για τις ημέρες της εβδομάδας*

Όπως φαίνεται στον κώδικα 11.9, η δήλωση απαριθμήσιμου τύπου γίνεται με χρήση της λέξης-κλειδί enum. Κάθε μια από τις μεταβλητές MONDAY έως FRIDAY είναι και μία σταθερά. Έτσι, τα αντίστοιχα αναγνωριστικά εκφράζονται με κεφαλαία ώστε να πληρούνται οι συμβάσεις ονοματολογίας των σταθερών. Επιπλέον, οι μεταβλητές αυτές είναι εξ ορισμού public και static. Από τη στιγμή που έχουμε διαθέσιμο τον τύπο Day, μπορούμε να τον χρησιμοποιήσουμε στον κώδικά μας, καταρχάς δηλώνοντας μεταβλητές τύπου Day, π.χ.

```
Day d=Day.FRIDAY;
```

Στη συνέχεια μπορούμε να χρησιμοποιήσουμε κανονικά τη μεταβλητή d, όπως για παράδειγμα δείχνει ο κώδικας που ακολουθεί:

```
switch (d) {
     case MONDAY:
          System.out.println("First day of the Week");
          break;
     case THUSDAY:
          System.out.println("Second day of the Week");
          break;
     case WEDNESDAY:
          System.out.println("It's a long Week");
          break;
     case THURSDAY:
          System.out.println("I fill tired");
          break;
     case FRIDAY:
          System.out.println("Last day of the Working Week");
          break;
     case SATURDAY:
          System.out.println("It's a terrific day");
          break;
     case SUNDAY:
          System.out.println("Rest today");
          break;
}
```

Ας δούμε όμως μερικά μυστικά αυτού του τύπου. Καταρχάς, πρόκειται για εξειδικευμένη κλάση. Οι δε τιμές MONDAY έως FRIDAY της Day δεν είναι παρά αντικείμενα της Day που έχουν δημιουργηθεί με τον default δημιουργό. Βεβαίως, όπως θα δούμε αμέσως παρακάτω, μπορούμε να ορίσουμε και δικούς μας δημιουργούς. Επιπλέον, τα αντικείμενα αυτά είναι public, static και final παρότι δεν έχουν δηλωθεί σαφώς ως τέτοια. Ο τύπος ονομάζεται απαριθμήσιμος καθώς τα αντικείμενα-τιμές αντιστοιχίζονται σε μία ακέραιη τιμή. Οι εξ ορισμού τιμές είναι 0,1,2,..,ν, για ν+1 τιμές ενός απαριθμήσιμου τύπου. Έτσι η τιμή της MONDAY είναι 0, της THUSDAY 1, κοκ, έως την SUNDAY με τιμή 6. Ο τύπος μάλιστα διαθέτει μια μέθοδο, την ordinal, που μας επιστρέφει την τιμή ενός απαριθμήσιμου αντικειμένου, π.χ. ο κώδικας

```
Day d = Day.FRIDAY;
System.out.println(d.ordinal());
```

θα τυπώσει 4.
Επιπλέον, η μέθοδος toString θα επιστρέψει σε μορφή String την ονομασία του αντικειμένου, δηλαδή ο κώδικας

```
Day d = Day.FRIDAY;
System.out.println(d);
```

εμφανίζει FRIDAY.
Επίσης, ο τύπος διαθέτει τη στατική μέθοδο values που επιστρέφει τα αντικείμενα σε πίνακα, δηλαδή η πρόταση

```
System.out.println(Arrays.asList(Day.values()));
```

θα εμφανίσει

```
[MONDAY, THUSDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
SUNDAY]
```

Τέλος, να επισημάνουμε πως ο εξ ορισμού δημιουργός του τύπου enum είναι ιδιωτικός ώστε να μην έχει ο χρήστης του την δυνατότητα να δημιουργήσει νέα αντικείμενα και να αλλοιώσει τον χαρακτήρα του τύπου.
Ας δούμε όμως και ένα άλλο παράδειγμα, όπου η αρίθμηση δεν ακολουθεί την τυπική περίπτωση από 0..ν. Στον κώδικα 11.10 δίνουμε έναν απαριθμήσιμο τύπο που αντιπροσωπεύει τους βαθμούς των καρτών μιας τράπουλας, έτσι ώστε το 2 της τράπουλας να έχει αρίθμηση 2, το 3 να έχει αρίθμηση 3 και το μοτίβο αυτό να ακολουθείται έως το 10, ενώ ο Βαλές έχει αρίθμηση 11, η Ντάμα 12, ο Ρήγας 13 και ο Άσος 14.

```
import java.util.Arrays;

public enum CardRank {

    TWO(2),
    THREE(3),
    FOUR(4),
    FIVE(5),
    SIX(6),
    SEVEN(7),
    EIGHT(8),
    NINE(9),
    TEN(10),
    JACK(11),
    QUEEN(12),
    KING(13),
    ACE(14);

    private final int cardValue;

    private CardRank(int value) {
        this.cardValue = value;
    }

    public int getCardValue() {
        return cardValue;
    }
}
```

Κώδικας 11.10 *Ο απαριθμήσιμος τύπος CardRank*

Καθώς δεν μας διευκολύνει η εξ ορισμού αρίθμηση, έχουμε φτιάξει έναν δημιουργό με τον οποίο δίνουμε τιμή στην cardValue την οποία μπορούμε να χρησιμοποιούμε σαν τιμή των μεταβλητών του τύπου. Κατά τη δημιουργία των αντικειμένων TWO έως ACE καλείται τώρα ο νέος δημιουργός και όπου παραστεί ανάγκη μπορούμε να πάρουμε ως αρίθμηση των αντικειμένων την cardValue μέσω της getCardValue. Προσοχή όμως, η τιμή αυτή δεν αντικαθιστά την τιμή που επιστρέφει η ordinal. Έτσι, η μεν CardRank.NINE.getCardValue() θα επιστρέψει 9, η δε CardRank.NINE.ordinal() θα επιστρέψει 7.

Ένα άλλο σημαντικό στοιχείο στον κώδικα 11.10 είναι πως ο δημιουργός είναι υποχρεωτικά ιδιωτικός. Αυτή είναι η τυπική περίπτωση στους απαριθμήσιμους τύπους. Κάνοντας τον δημιουργό ιδιωτικό, μπορεί να χρησιμοποιηθεί μόνο από την κλάση για την δημιουργία των απαιτούμενων τιμών, δηλαδή στο παράδειγμά μας, για τη δημιουργία των σταθερών TWO έως ACE. Αν όμως είχαμε ορίσει έναν δημιουργό δημόσιο, τότε οποιοσδήποτε θα μπορούσε να δημιουργήσει τέτοια αντικείμενα με αποτέλεσμα να καταστραφεί η έννοια του απαριθμήσιμου τύπου. Φανταστείτε να δημιουργήσει κανείς τιμή FRIDAY τύπου CardRank. Αν προσπαθήσετε να δηλώσετε δημιουργό σε τύπο enum με δημόσια ή προστατευμένη πρόσβαση, ο μεταγλωττιστής θα διαμαρτυρηθεί. Αν πάλι δεν δηλώσετε προσδιοριστή πρόσβασης, τότε η πρόσβαση δεν θα είναι πρόσβαση πακέτου όπως ισχύει για τις τυπικές κλάσεις αλλά αντίθετα θα καθορισθεί άρρητα ως ιδιωτική.

Έχοντας τον δημιουργό ιδιωτικό έχουμε εξασφαλίσει πως μέσα στο πρόγραμμά μας δεν υπάρχουν άλλα αντικείμενα τύπου CardRank πέραν των δηλωθέντων στον ίδιο τον τύπο. Επομένως, κάθε βαθμός κάρτας στο πρόγραμμά μας είναι μοναδικός, δηλαδή υπάρχει ένα μόνο αντικείμενο TWO, ένα μόνο αντικείμενο THREE και ούτω καθεξής. Σαν συνέπεια, μπορούμε να ελέγχουμε για ισότητα τα αντικείμενα CardRank με χρήση του τελεστή ισότητας. Παρόλο που ο τύπος περιλαμβάνει τη συνάρτηση equals, σε αυτήν την περίπτωση δεν είναι αναγκαία η χρήση της. Αν υποθέσουμε πως έχουμε δύο αναφορές σε ένα αντικείμενο CardRank. Αν και οι δύο αναφορές έχουν ως περιεχόμενο το αντικείμενο TWO ας πούμε, τότε θα αναφέρονται στην ίδια διεύθυνση, μιας και υπάρχει μόνο ένα τέτοιο στιγμιότυπο στην εφαρμογή μας. Επομένως, η σύγκριση με τον τελεστή ισότητας είναι έγκυρη και με δεδομένο πως είναι ταχύτερη από την κλήση της equals την προτιμούμε.

11.8 Εμφωλευμένες κλάσεις

Έχουμε εξηγήσει πως σε ένα αρχείο java μπορεί να περιέχεται μία μόνο δημόσια κλάση και μάλιστα απαιτείται το όνομα του αρχείου να είναι ίδιο με το όνομα της κλάσης (ενότητα 2.2). Τίποτα όμως δεν μας εμποδίζει να τοποθετήσουμε σε ένα αρχείο περισσότερες από μία μη δημόσιες κλάσεις. Για παράδειγμα, ο κώδικας

```
class NonPublic { }

public class NonPublicDemo {

     public static void main(String[] args) {
          NonPublic nP=new NonPublic();
     }
}
```

έστω ότι περιλαμβάνει την κλάση NonPublic στο ίδιο αρχείο με τη δημόσια κλάση NonPublicDemo. Σε αυτό το επίπεδο επιτρέπεται μόνο η πρόσβαση πακέτου για την κλάση NonPublic.
Υπάρχει, επιπλέον η δυνατότητα να ορίσουμε εμφωλευμένες κλάσεις (nested class). Οι εμφωλευμένες κλάσεις είναι δύο κατηγοριών, οι στατικές και οι μη στατικές που ονομάζονται εσωτερικές (inner). Ο κώδικας που ακολουθεί επιδεικνύει τη χρήση εσωτερικής κλάσης.

```
public class Outer {

     private int k = 1;

     class Inner {

          int getK() {
               return k;
          }
```

```
        }

        public static void main(String[] args) {
              Outer outer = new Outer();
              Outer.Inner iCObject = outer.new Inner();
              System.out.println(outer.k + " " + iCObject.getK());
        }
  }
```

Οι εσωτερικές κλάσεις δέχονται όλους τους ρυθμιστές πρόσβασης, δηλαδή μία εσωτερική κλάση μπορεί να χαρακτηριστεί ως δημόσια, ιδιωτική, προστατευμένη (ενότητα 13.1.2) ή απλώς να διατηρήσει την πρόσβαση πακέτου. Ένα πλεονέκτημα των εσωτερικών κλάσεων είναι ότι έχουν πρόσβαση στα μέλη της κλάσης που ανήκουν ακόμη και όταν αυτά είναι ιδιωτικά. Όπως φαίνεται στον κώδικα, η κλάση Inner προσπελαύνει το ιδιωτικό μέλος k της Outer. Για να δημιουργηθεί αντικείμενο εσωτερικής κλάσης, πρέπει πρώτα να δημιουργηθεί αντικείμενο της περιβάλλουσας μέσα από το οποίο μπορεί να κληθεί ο τελεστής new όπως επιδεικνύει ο παραπάνω κώδικας. Οι εσωτερικές κλάσεις μπορούν να αναπτυχθούν και τοπικά σε επίπεδο συνάρτησης.

Ένας άλλος τύπος εσωτερικής κλάσης είναι η ανώνυμη κλάση (Anonymous class). Θα παρουσιάσουμε την ανώνυμη κλάση στην ενότητα 14.4.5.

Όπως όλα τα μέλη μιας κλάσης, έτσι και τα μέλη-κλάσεις μπορούν να είναι στατικά. Ακολουθεί παράδειγμα χρήσης εμφωλευμένης στατικής κλάσης (nested static class):

```
public class Outer {

     private int k = 1;

     static class NestedStatic {

          int getK(Outer outer) {
               return outer.k;
          }
     }

     public static void main(String[] args) {
          Outer.NestedStatic nS = new NestedStatic();
          Outer outer = new Outer();
          System.out.println(outer.k + " " + nS.getK(outer));
     }
}
```

Η κλάση NestedStatic έχει δηλωθεί ως στατική. Γενικά τα στατικά μέλη δεν έχουν πρόσβαση στα μη στατικά μέλη μιας κλάσης. Ο γενικός αυτός κανόνας ισχύει και για τις κλάσεις. Ένα στατικό μέλος μπορεί να εκτελεστεί αν είναι μέθοδος ή να δημιουργηθεί αν είναι κλάση χωρίς να συσχετίζεται υποχρεωτικά με κάποιο στιγμιότυπο της κλάσης. Επομένως, δεν είναι δυνατόν να προσπελάσει άμεσα τα μη στατικά μέλη. Μπορεί βέβαια η προσπέλαση να επιτευχθεί μέσα από ένα αντικείμενο της κλάσης. Έτσι, η getK εδώ λαμβάνει ως παράμετρο ένα αντικείμενο outer του οποίου εμφανίζει την μεταβλητή k. Όπως οι εσωτερικές κλάσεις, έτσι και οι εμφωλευμένες στατικές μπορούν να συνταχθούν με οποιονδήποτε προσδιοριστή πρόσβασης.

11.9 Λυμένες Ασκήσεις

Παρουσιάζουμε εδώ μια σειρά από λυμένες ασκήσεις που θα βοηθήσουν τον αναγνώστη να εξοικειωθεί με τη λογική της ενσωμάτωσης.

11.9.1 Η κλάση Person

Να υλοποιηθεί η κλάση Person. Κάθε Person διαθέτει όνομα, επώνυμο και φύλο. Για το φύλο χρησιμοποιήστε int έτσι ώστε το 0 να αναπαριστά το γυναικείο φύλλο και το 1 το ανδρικό. Ορίστε τις αναγκαίες συναρτήσεις.

Λύση

```
public class Person {

    private String fName;
    private String sName;
    private int sex;

    public Person(String fName, String sName, int sex) {
        this.fName = fName;
        this.sName = sName;
        this.sex = sex;
    }

    public String getfName() {
        return fName;
    }

    public void setfName(String fName) {
        this.fName = fName;
    }

    public String getsName() {
        return sName;
    }

    public void setsName(String sName) {
        this.sName = sName;
    }

    public int getSex() {
        return sex;
    }

    public void setSex(int sex) {
        this.sex = sex;
    }

    @Override
    public int hashCode() {
        int hash = 5;
        hash = 89 * hash + this.fName.hashCode();
        hash = 89 * hash + this.sName.hashCode();
        hash = 89 * hash + this.sex;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
```

```
            final Person other = (Person) obj;
            if (this.sex != other.sex) {
                return false;
            }
            if (!this.fName.equals(other.fName)) {
                return false;
            }
            return this.sName.equals(other.sName);
        }

        @Override
        public String toString() {
            return "Person{" + "fName=" + fName + ", sName=" + sName + ", sex="
    + (sex==1?"M":"F") + '}';
        }
    }
```

Κώδικας 11.11 *Η κλάση Person*

Στον κώδικα 11.11 παρατίθεται η κλάση Person. Πρόκειται για απλή εφαρμογή της ενσωμάτωσης. Παρόμοιο αποτέλεσμα μπορεί να παραχθεί αυτόματα από το NetBeans ως εξής: Δημιουργήστε μια κλάση Person και προσθέστε τις μεταβλητές στιγμιότυπου. Μεταβείτε στον editor του NetBeans στην κλάση. Με δεξί κλικ ανοίγει ένα μενού. Επιλέξτε Insert Code. Από το υπομενού που θα ανοίξει επιλέξτε σταδιακά την αυτόματη δημιουργία δημιουργών, αναγνωστών, ρυθμιστών, μεθόδου equals, μεθόδου hashcode και μεθόδου toString. Εντοπίστε και μελετήστε τις διαφορές μεταξύ του κώδικα που παράγει το NetBeans και του κώδικα 11.11.

11.9.2 Η κλάση MyInteger

Υλοποιήστε την κλάση MyInteger που περιλαμβάνει τις συναρτήσεις που περιγράφονται παρακάτω και όλες τις αναγκαίες συναρτήσεις.

```
public class MyInteger {
     private int value;
     public MyInteger();
     public MyInteger(int k);
     public MyInteger add(MyInteger i);
     public MyInteger subtract(MyInteger i);
     public MyInteger multiply(MyInteger i);
     public MyInteger divide(MyInteger i);
     public String toString();
     public boolean equals(Object obj);
     public int hashCode();
};
```

Να υπερφορτωθούν οι μέθοδοι setValue, add, subtract, multiply, divide ώστε να δέχονται παραμέτρους τύπου Integer και τύπου int.

Λύση

```
public class MyInteger {

    private int value;

    public MyInteger() {
        value = 0;
    }

    public MyInteger(int k) {
        value = k;
```

```
    }

    public void setValue(int k) {
        value = k;
    }

    public void setValue(Integer k) {
        value = k;
    }

    public void setValue(MyInteger k) {
        value = k.value;
    }

    public int getValue() {
        return value;
    }

    public MyInteger add(MyInteger i) {
        return new MyInteger(value + i.value);
    }

    public MyInteger subtract(MyInteger i) {
        return new MyInteger(value - i.value);
    }

    public MyInteger multiply(MyInteger i) {
        return new MyInteger(value * i.value);
    }

    public MyInteger divide(MyInteger i) {
        return new MyInteger(value / i.value);
    }

    @Override
    public String toString() {
        return Integer.toString(value);
    }

    @Override
    public int hashCode() {
        int hash = 7;
        hash = 61 * hash + this.value;
        return hash;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final MyInteger other = (MyInteger) obj;
        return this.value == other.value;
    }
}
```

Κώδικας 11.12 *Η κλάση MyInteger*

Η κλάση MyInteger (κώδικας 11.12) εκτός από τις αναγκαίες συναρτήσεις περιλαμβάνει και μια σειρά από συναρτήσεις που υποστηρίζουν τις βασικές πράξεις. Η υλοποίησή τους είναι πολύ απλή. Σε σχέση με την απαίτηση της εκφώνησης να υπερφορτωθούν οι συναρτήσεις setValue, add, subtract, multiply, divide, έχουμε υλοποιήσει την υπερφόρτωση μόνο για την setValue. Παρόμοια μπορείτε να υλοποιήσετε και τις υπόλοιπες αναγκαίες μεθόδους.

11.9.3 Μονάδα Μέτρησης Βάρους

Να υλοποιηθεί η κλάση WeightMeasurementUnit (Μονάδα Μέτρησης Βάρους). Η κλάση διαθέτει δύο μεταβλητές στιγμιότυπου, την description τύπου String που αναπαριστά το λεκτικό της μονάδας και την relationToGr τύπου double που αναπαριστά τη σχέση της μονάδας με το γραμμάριο. Να προστεθεί η συνάρτηση main στην οποία να δημιουργηθούν τρία αντικείμενα, ένα που αναπαριστά το χιλιόγραμμο, ένα για τον τόνο και ένα για το χιλιοστό του γραμμαρίου.

Λύση

```
public class WeightMeasurementUnit {

    private String description;
    private double relationToGr;

    public WeightMeasurementUnit(String lekt, double relationToGr) {
        if (relationToGr <= 0) {
            throw new RuntimeException();
        }
        this.description = lekt;
        this.relationToGr = relationToGr;
    }

    public String getDescription() {
        return description;
    }

    public double getRelationToGr() {
        return relationToGr;
    }

    public void setDescription(String description) {
        this.description = description;
    }


    @Override
    public int hashCode() {
        return Double.hashCode(this.relationToGr);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
```

```
        }
        final WeightMeasurementUnit other = (WeightMeasurementUnit) obj;
        if (Double.compare(this.relationToGr, other.relationToGr) != 0) {
            return false;
        }
        return true;
    }

    @Override
    public String toString() {
        return "WeightMeasurementUnit{" + "description=" + description + ",
relationToGr=" + relationToGr + '}';
    }

    public static void main(String[] args) {
        WeightMeasurementUnit kgr = new WeightMeasurementUnit("Kgr", 1000),
                metricTon = new WeightMeasurementUnit("Metric Ton", 1000000),
                mgr = new WeightMeasurementUnit("mgr", 1 / 1000d);
        System.out.println(kgr);
        System.out.println(metricTon);
        System.out.println(mgr);
    }
}
```

Κώδικας 11.13 *Κλάση αναπαράστασης της Μονάδας Μέτρησης Βάρους*

Ο κώδικας 11.13 δίνει τον ορισμό της κλάσης WeightMeasurementUnit. Κάθε τέτοια μονάδα προσδιορίζεται από τη σχέση της με το γραμμάριο. Έτσι στον δημιουργό της κλάσης γίνεται έλεγχος ώστε η σχέση της μονάδας με το γραμμάριο να είναι έγκυρη. Στη συνέχεια λαμβάνουν τιμές τα πεδία της κλάσης και δημιουργείται το αντικείμενο. Η ισότητα βασίζεται αποκλειστικά στη μεταβλητή relationToGr, δηλαδή δύο μονάδες μέτρησης βάρους με τη ίδια σχέση με το γραμμάριο θεωρούνται ίσες ανεξάρτητα από τα υπόλοιπα χαρακτηριστικά τους. Αντίστοιχα και η hashCode βασίζεται αποκλειστικά στηνσχέση με το γραμμάριο.

11.9.4 Βάρος

Να υλοποιηθεί η κλάση Weight. Κάθε αντικείμενο της κλάσης διαθέτει δύο μεταβλητές. Η μια εκφράζει το μέτρο του βάρους και είναι double και η άλλη τη μονάδα μέτρησης του βάρους και είναι WeightMeasurementUnit.
Η κλάση διαθέτει τρεις δημιουργούς ως εξής:

```
public Weight(double value, WeightMeasurementUnit mB)
```

Δημιουργεί αντικείμενο με μέτρο value και μονάδα μέτρησης mB.

```
public Weight(Weight w)
```

Δημιουργεί αντίγραφο του w.

```
public Weight(double value, String unitDescription, double
relationToGr)
```

Δημιουργεί αντικείμενο με μέτρο value και μονάδα μέτρησης με περιγραφή unitDescription και σχέση με το γραμμάριο relationToGr.

Πέραν των αναγκαίων συναρτήσεων, η κλάση διαθέτει και τις ακόλουθες συναρτήσεις:

```
public static Weight convert(WeightMeasurementUnit from,
WeightMeasurementUnit to)
```

Μετατρέπει τη μονάδα μέτρησης from μονάδα μέτρησης to. Προσέξτε πως επιστρέφει Weight. Πράγματι, αν για παράδειγμα μετατρέψω το κιλό σε γραμμάρια θα λάβω ως αποτέλεσμα 1000 γραμμάρια, μέγεθος που

υποχρεωτικά εκφράζεται σε βάρος. Η συνάρτηση είναι στατική καθώς δεν είναι απαραίτητο να διαθέτουμε αντικείμενο Weight για να μετατρέψουμε μια μονάδα σε μία άλλη.

```
public Weight convert(WeightMeasurementUnit wUnit)
```

Μετατρέπει αυτό το βάρος στην μονάδα wUnit.

```
public Weight add(Weight w)
```

Προσθέτει αυτό το βάρος, δηλαδή το βάρος που την καλεί, με το βάρος w. Επιστρέφει το άθροισμα εκφρασμένο στη μονάδα βάρους που καλεί την add.

Λάβετε υπόψη πως η μέθοδος equals θα πρέπει να επιστρέφει true για ίσα βάρη. Για παράδειγμα, το 1 κιλό είναι ίσο με τα 1000 γραμμάρια. Αντίστοιχα, και η μέθοδος hashCode πρέπει να επιστρέφει την ίδια τιμή για βάρη που είναι ίσα.

Προσθέστε στην κλάση, αυτήν την main

```
public static void main(String[] args) {
     WeightMeasurementUnit          kgr          =          new
     WeightMeasurementUnit("kgr", 1000);
     WeightMeasurementUnit          mgr          =          new
     WeightMeasurementUnit("mgr", 1 / 1000d);
     Weight w1 = new Weight(1, kgr);
     Weight w2 = new Weight(100, mgr);
     Weight w3 = w1.add(w2);
     Weight w4 = w2.add(w1);
     System.out.println(w3);
     System.out.println(w4);
     System.out.println(w3.equals(w4));
     System.out.println(w3.hashCode() == w4.hashCode());
}
```

Ελέγξτε ώστε η έξοδος να είναι σύμφωνη με

```
1.0001 kgr
1000100.0 mgr
true
true
```

Λύση

```
public class Weight {

    private double value;
    private WeightMeasurementUnit weightUnit;

    public Weight(double value, String unitDescription,
                  double relationToGr) {
        this(value, new WeightMeasurementUnit(unitDescription,
                  relationToGr));
    }

    public Weight(double value, WeightMeasurementUnit mB) {
        this.value = value;
        this.weightUnit = mB;
    }

    public Weight(Weight w) {
        this(w.value, w.weightUnit);
    }
```

```
public static Weight convert(WeightMeasurementUnit from,
        WeightMeasurementUnit to) {
    double valueTo = from.getRelationToGr() / to.getRelationToGr();
    return new Weight(valueTo, to);
}

public Weight convert(WeightMeasurementUnit wUnit) {
    Weight rVal = convert(weightUnit, wUnit);
    rVal.value *= value;
    return rVal;
}

public Weight add(Weight w) {
    //Weight w1 = new Weight(w);
    if (!weightUnit.equals(w.weightUnit)) {
        w = w.convert(weightUnit);
    }
    return new Weight(value + w.value, weightUnit);
}

public void setValue(double value) {
    this.value = value;
}

public double getValue() {
    return value;
}

public WeightMeasurementUnit getWeightUnit() {
    return weightUnit;
}

public void setWeightUnit(WeightMeasurementUnit weightUnit) {
    this.weightUnit = weightUnit;
}

@Override
public int hashCode() {
    Weight w = convert(new WeightMeasurementUnit("gr", 1));
    int hash = 7;
    hash = 23 * hash + Double.hashCode(w.value);
    return hash;
}

@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    Weight other = (Weight) obj;
    other = other.convert(this.weightUnit);
    return Double.compare(this.value, other.value) == 0;
}
```

```
    @Override
    public String toString() {
        return value + " " + weightUnit.getDescription();
    }

    public static void main(String[] args) {
        WeightMeasurementUnit kgr = new WeightMeasurementUnit("kgr", 1000);
        WeightMeasurementUnit mgr = new WeightMeasurementUnit("mgr", 1 / 1000d);
        Weight w1 = new Weight(1, kgr);
        Weight w2 = new Weight(100, mgr);
        Weight w3 = w1.add(w2);
        Weight w4 = w2.add(w1);
        System.out.println(w3);
        System.out.println(w4);
        System.out.println(w3.equals(w4));
        System.out.println(w3.hashCode() == w4.hashCode());
    }
}
```

Κώδικας 11.14 *Η κλάση Weight*

Για να μετατρέψω μια μονάδα βάρους σε μια άλλη θα πρέπει να αξιοποιήσω την σχέση κάθε μονάδας με το γραμμάριο. Έστω ότι θέλω να μετατρέψω τη μονάδα βάρους from στην to. Η from σε γραμμάρια είναι ένα βάρος με τιμή from.relationToGr και μονάδα το γραμμάριο. Αντίστοιχα, η to σε γραμμάρια είναι ένα βάρος με μέτρο to.relationToGr και μονάδα το γραμμάριο. Τώρα και οι δύο μονάδες είναι εκφρασμένες σε βάρη με μονάδα το γραμμάριο. Επομένως, η διαίρεση from.relationToGr/to.relationToGr μας δίνει το μέτρο του βάρους που εκφράζει τη μονάδα from στην μονάδα to. Αυτή είναι η λογική της convert που μετατρέπει από μια μονάδα σε άλλη, δηλαδή της convert(WeightMeasurementUnit, WeightMeasurementUnit) του κώδικα 11.14.

Για να μετατρέψω τη μονάδα μέτρησης ενός βάρους, στη μονάδα μέτρησης, έστω wUnit, μετατρέπω πρώτα τη μονάδα μέτρησης του βάρους, δηλαδή την weightUnit στην wUnit. Στη συνέχεια, το μέτρο του βάρους που λαμβάνω ως αποτέλεσμα της μετατροπής των μονάδων, το πολλαπλασιάζω με το μέτρο αυτού του βάρους. Σε αυτήν τη λογική βασίζεται η convert(WeightMeasurementUnit) του κώδικα 11.14.

Στη συνάρτηση add ελέγχω αν τα βάρη που πρόκειται να προστεθούν εκφράζονται στην ίδια μονάδα βάρους. Αν όχι μετατρέπω το βάρος που είναι παράμετρος της add στη μονάδα του βάρους που καλεί την add. Εφόσον τα δύο βάρη εκφράζονται στην ίδια μονάδα, η πρόσθεσή τους γίνεται απλώς με την πρόσθεση των μέτρων τους. Το νέο βάρος έχει ως μονάδα την μονάδα του βάρους που κάλεσε την add. Έτσι, η κλήση w1.add(w2) δεν θα επιστρέψει ακριβώς το ίδιο αποτέλεσμα με την κλήση w2.add(w1), καθώς στην πρώτη περίπτωση το βάρος εκφράζεται στη μονάδα της w1 και στη δεύτερη στη μονάδα της w2. Ωστόσο, τα δύο αυτά αθροίσματα παρότι εκφράζονται σε διαφορετικές μονάδες βάρους, αντιπροσωπεύουν ίσα βάρη, πράγμα που πρέπει να επιβεβαιώνεται και από τη μέθοδο equals της κλάσης.

Η μέθοδος equals θα πρέπει να επιστρέφει true για βάρη που είναι ίσα μεταξύ τους, π.χ. το 1 kgr με τα 1000 gr, πρέπει να θεωρούνται ίσα. Έτσι μετατρέπουμε την παράμετρό της στη μονάδα βάρους του αντικειμένου που την καλεί και στη συνέχεια συγκρίνουμε τα μέτρα των δύο βαρών.

Παρόμοια, η hashCode πρέπει να επιστρέφει τον ίδιο κωδικό για δύο βάρη εφόσον η equals τα θεωρεί ίσα. Έτσι μετατρέπουμε κάθε βάρος σε γραμμάρια και παράγουμε κωδικό βασισμένοι στον αριθμό των γραμμαρίων ενός βάρους.

11.9.5 Η κλάση Complex

Κάθε μιγαδικός αριθμός αποτελείται από ένα ζεύγος πραγματικών και συμβολίζεται ως (a,b) ή a+bi, όπου ο πραγματικός a αποτελεί το πραγματικό μέρος του μιγαδικού και ο b το φανταστικό μέρος.
Η πρόσθεση μεταξύ δύο μιγαδικών δίνεται από τον τύπο

$$(x + yi) + (u + vi) = (x + u) + (y + v)i$$

Η αφαίρεση δίνεται από τον τύπο

$$(x + yi) - (u + vi) = (x - u) + (y - v)i$$

Ο πολλαπλασιασμός από τον τύπο

$$(x + yi)(u + vi) = (xu - yv) + (xv - yu)i$$

Ο αντίστροφος ενός μιγαδικού δίνεται από τον τύπο

$$\frac{1}{x + yi} = \frac{x}{x^2 + y^2} - \frac{y}{x^2 + y^2}i$$

Να υλοποιηθεί η κλάση Complex των μιγαδικών αριθμών. Να προστεθούν μέθοδοι για την πρόσθεση, την αφαίρεση, τον πολλαπλασιασμό και τη διαίρεση μιγαδικών. Να υποστηρίζονται και οι δύο συμβολισμοί των μιγαδικών. Να προστεθεί συνάρτηση main στην οποία να διαιρείται ο 2+3i με τον 4+0i και ο 20-4i με τον 3+2i.

Σημείωση: Τα αποτελέσματα των ζητούμενων διαιρέσεων είναι 0.5+0.75i και 4.0-4.0i, αντίστοιχα.

Λύση

```
public class Complex {

    public static int format = 1;
    private double r;
    private double f;

    public Complex(double r, double f) {
        this.r = r;
        this.f = f;
    }

    public Complex(Complex c) {
        this(c.r, c.f);
    }

    public double getR() {
        return r;
    }

    public void setR(double r) {
        this.r = r;
    }

    public double getF() {
        return f;
    }

    public void setF(double f) {
        this.f = f;
    }

    @Override
    public int hashCode() {
        return Double.hashCode(r) + Double.hashCode(f);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
```

```
    if (obj == null) {
        return false;
    }
    if (getClass() != obj.getClass()) {
        return false;
    }
    final Complex other = (Complex) obj;
    if (Double.compare(this.r, other.r) != 0) {
        return false;
    }
    return Double.compare(this.f, other.f) == 0;
}

@Override
public String toString() {
    if (r==0 && f==0) return "0";
    String rVal="";
    if (format == 1) {
        if (r!=0)
            rVal+=r;  //8i, 2-81
        if (f>0)
            rVal+=(r!=0?"+":"")+f+"i";
        else
            if (f<0)
                rVal+=f+"i";
    }
    else {
        rVal="("+r+","+f+")";
    }
    return rVal;
}

public Complex add(Complex c) {
    double real=r+c.r;
    double imagin=f+c.f;
    return new Complex(real, imagin);
}

public Complex subtract(Complex c) {
    double real=r-c.r;
    double imagin=f-c.f;
    return new Complex(real, imagin);
}

public Complex multiply(Complex c) {
    double real=r*c.r-f*c.f;
    double imagin=r*c.f+f*c.r;
    return new Complex(real, imagin);
}

public Complex divide(Complex c) {
    if (c.r==0 && c.f==0)
        throw new RuntimeException();
    double real=c.r/(Math.pow(c.r, 2)+Math.pow(c.f, 2));
    double imagin=c.f/(Math.pow(c.r, 2)+Math.pow(c.f, 2));
    Complex divisor= new Complex(real,-imagin);
    return multiply(divisor);
}

public static void main(String[] args) {
    Complex.format=1;
```